

Multithreading



What is a process?

A **process** is an independent program in execution. It has the following characteristics:

- 1.Memory Isolation:** A process has its own dedicated memory space (address space) and resources. This ensures that processes are isolated from one another.
- 2.Independent Execution:** Processes execute independently of each other, and communication between them requires inter-process communication (IPC) mechanisms like pipes, sockets, or shared memory.
- 3.Multiple Threads:** A process can contain multiple threads, which are the smaller units of execution.
- 4.Heavyweight:** Creating and managing processes is more resource-intensive compared to threads because each process has its own resources like memory and file handles.



Thread in Multithreading

- A **thread** is the smallest unit of execution within a process. In a multithreaded application:
- A single process can have multiple threads running concurrently.
- Threads within the same process share the same memory and resources (e.g., variables, open files).
- Threads enable parallelism and faster execution on multi-core processors.



Example: Process vs. Thread

Imagine you have a text editor as a **process**:

- The process itself handles the overall application (e.g., opening files).
- Threads within the process might perform specific tasks like spell checking, autosaving, or rendering the UI concurrently.



Benefits of multithreading in concurrent programming.

1. Improved Performance Through Parallelism

- **Benefit:** Multithreading allows multiple tasks to run simultaneously, leveraging multi-core processors to improve execution speed.
- **Example:** In a video editing application:
 - One thread processes video frames.
 - Another thread handles audio processing.
 - A third thread renders the preview. This parallelism ensures smooth and faster video rendering.



Benefits of multithreading in concurrent programming.

2. Efficient Utilization of CPU Resources

- **Benefit:** Threads keep the CPU busy by executing tasks while other threads are waiting for I/O operations (e.g., file reading, network requests).
- **Example:** In a web server:
 - One thread handles incoming HTTP requests.
 - Another thread reads from the database.
 - A third thread sends responses to the clients. This ensures that the server does not waste time idling during I/O operations.



Benefits of multithreading in concurrent programming.

3. Responsive Applications

- **Benefit:** Multithreading allows an application to remain responsive even when performing long or intensive tasks.
- **Example:** In a graphical user interface (GUI) application:
 - The main thread manages the UI.
 - A background thread performs file uploads or calculations. This ensures the application does not freeze while waiting for the task to complete.



Benefits of multithreading in concurrent programming.

4. Concurrent Handling of Independent Tasks

- **Benefit:** Tasks that are logically independent can be executed concurrently without blocking each other.
- **Example:** In an e-commerce website:
 - One thread fetches product details.
 - Another processes payment transactions.
 - A third sends order confirmation emails. All these tasks run independently, enhancing user experience.



Scheduler

- A **scheduler** in multithreading is a component of the operating system (OS) or runtime environment that manages the execution of threads by allocating CPU time to them.
- It determines the order and duration of thread execution to efficiently utilize the CPU and ensure fairness among threads.



Functions of a Scheduler

- **1.Thread Management:**Keeps track of all threads, their states, and readiness to execute.CPU
- **2.Time Allocation:**Determines which thread gets access to the CPU and for how long.
- **3. Context Switching:**Handles switching between threads when a running thread is paused or completed.
- **4. Fairness:**Ensures no thread is unfairly starved of CPU time (though this depends on the scheduling policy).



Thread life cycle

- **1. New (Created):**
- When a thread is created using a programming construct (e.g., Thread class in Java), it is in the new state. At this stage, the thread object exists but has not started running.

Example :

```
Thread thread = new Thread();
```

- The thread is created but not yet started



Thread life cycle

- **2. Runnable (Ready to Run):**

- When the start() method is called on a thread, it moves to the runnable state.
- The thread is ready to run and waiting for CPU time but is not yet running. It depends on the thread scheduler to allocate CPU time to the thread.
- Example:
- `thread.start();` // Moves the thread to runnable



Thread life cycle

- **3. Running:**

- A thread is in the running state when it is actively executing.
- This state occurs when the thread scheduler selects the thread from the runnable pool and allocates CPU time to it.
- Note: A thread can move back to the runnable state if the CPU time is taken away before it completes.



Thread life cycle

- **4. Blocked (Waiting/Timed Waiting)**

- A thread is **blocked** if it cannot proceed due to some condition (e.g., waiting for input/output, waiting to acquire a lock, or sleeping for a specific time).
- Types of blocking:
 - **Waiting**: Indefinitely waiting for another thread or condition.
 - **Timed Waiting**: Waiting for a fixed amount of time.

Example:

`Thread.sleep(1000);` // Thread goes into timed waiting for 1 second



Thread life cycle

- **5. Terminated (Dead)**

- A thread enters the **terminated** state when it has completed its task or is explicitly stopped.
- Once terminated, a thread cannot be restarted.

Example:

`thread.join();` // Waits for the thread to complete its execution



Concurrency vs parallelism

- **Concurrency**

1. Definition:

1. Concurrency refers to the ability to manage multiple tasks **at the same time**, but not necessarily performing them simultaneously.
2. Tasks can make progress **independently** but may share the same CPU core through time-slicing.

2. Execution:

1. Achieved through **context switching**, where the CPU alternates between tasks, giving the illusion of simultaneous execution.
2. Primarily useful in systems with a **single core**.



Concurrency vs parallelism

- **Concurrency (continued)**

3.Example:

- A program switches between downloading a file and processing user input. Both tasks make progress, but only one task runs at a time.

4.Key Focus:

- Handling **many tasks** at once.
- Improves responsiveness by avoiding blocking (e.g., a GUI remains responsive while processing data).

5. Analogy:

- Think of a single person multitasking, such as answering calls and writing emails by switching between them.



Concurrency vs parallelism

- **Parallelism**

1. Definition:

1. Parallelism involves performing **multiple tasks simultaneously** using multiple CPU cores or processors.
2. True parallelism requires **hardware support** like multi-core CPUs or distributed systems.

2. Execution:

1. Multiple threads or processes execute at the **same time** on different cores.
2. Only possible on systems with **multiple cores or processors**.

3. Example:

- A program calculates the sum of two large datasets using two threads, where each thread runs on a separate core simultaneously.



Concurrency vs parallelism

- **Parallelism(continued)**
- **4.Key Focus:**
 - Achieving simultaneous execution to improve performance and throughput.
- **5.Analogy:**
 - Think of a group of people working on different parts of a project simultaneously.



Multithreading

Multithreading is the ability of a program or process to execute multiple threads concurrently, where each thread represents a separate path of execution. Threads share the same memory space within a process but can run independently, enabling tasks to be performed simultaneously or in parallel.



Advantages of Multithreading

1.Improved Performance:

- Utilizes multiple CPU cores for faster execution of tasks.

2.Responsiveness:

- Keeps applications responsive by delegating long-running operations to separate threads (e.g., GUI apps).

3.Resource Sharing:

- Threads share memory and resources, making inter-thread communication easier and faster.

4.Parallelism:

- Allows tasks to be divided into smaller sub-tasks that can run in parallel on multiple processors.



Disadvantages of Multithreading

- **1.Complexity:** Writing and debugging multithreaded programs is more complex due to potential issues like deadlocks, race conditions, and thread starvation.
- **2.Overhead:** Context switching between threads can lead to overhead if not managed efficiently.
- **3. Synchronization:** Requires careful management of shared resources to avoid data inconsistency or corruption.



Java's multithreading mechanisms:

- Threads can be created by using two mechanisms :
- Extending the Thread class
- Implementing the Runnable Interface



Thread creation by extending the Thread class

- We create a class that extends the **java.lang.Thread** class. This class overrides the run() method available in the Thread class.
- A thread begins its life inside run() method. We create an object of our new class and call start() method to start the execution of a thread. Start() invokes the run() method on the Thread object



Example of Thread creation by using the Thread class

```
package rw.ac.rac;

public class Boys extends Thread {
    public void run() {
        for(int i=0;i<100;i++) {
            System.out.println("Boy");
            try {
                Thread.sleep(10);
            } catch (InterruptedException e) {
                // TODO Auto-generated catch block
                e.printStackTrace();
            }
        }
    }
}

package rw.ac.rac;

public class SittingMain {

    public static void main(String[] args) {
        Boys boy=new Boys();
        Girls girl=new Girls();
        boy.start();
        try {
            Thread.sleep(5);
        } catch (InterruptedException e) {
            // TODO Auto-generated catch block
            e.printStackTrace();
        }
        girl.start();
    }
}
```

```
package rw.ac.rac;

public class Girls extends Thread{
    public void run() {
        for(int i=0;i<100;i++) {
            System.out.println("Girl");
            try {
                Thread.sleep(10);
            } catch (InterruptedException e) {
                // TODO Auto-generated catch block
                e.printStackTrace();
            }
        }
    }
}
```



Thread creation by implementing the Runnable Interface

- We create a new class which implements `java.lang.Runnable` interface and override `run()` method. Then we instantiate a `Thread` object and call `start()` method on this object.
- If we extend the `Thread` class, our class cannot extend any other class because Java doesn't support multiple inheritance. But, if we implement the `Runnable` interface, our class can still extend other base classes.



Thread creation by implementing the Runnable Interface

```
package yearTwo;

public class Boys implements Runnable {

    @Override
    public void run() {
        for(int i=0;i<100;i++) {
            System.out.println("Boy");
            try {
                Thread.sleep(10);
            } catch (InterruptedException e) {
                // TODO Auto-generated catch block
                e.printStackTrace();
            }
        }
    }

}

package yearTwo;

public class SittingMain {

    public static void main(String[] args) {
        Runnable boy=new Boys();
        Runnable girl=new Girls();
        Thread t1=new Thread(boy);
        Thread t2=new Thread(girl);
        t1.start();
        t2.start();
    }

}
```

```
package yearTwo;

public class Girls implements Runnable {

    @Override
    public void run() {
        for(int i=0;i<100;i++) {
            System.out.println("Girl");
            try {
                Thread.sleep(10);
            } catch (InterruptedException e) {
                // TODO Auto-generated catch block
                e.printStackTrace();
            }
        }
    }

}
```

Thread creation by implementing the Runnable Interface(Use of lambda expression)

```
public class SittingMain {  
  
    public static void main(String[] args) {  
        Thread t1=new Thread(  
            ()->{for(int i=0;i<100;i++) {  
                System.out.println("Boy");  
                try {  
                    Thread.sleep(10);  
                } catch (InterruptedException e) {  
                    // TODO Auto-generated catch block  
                    e.printStackTrace();  
                }  
            }  
        );  
        Thread t2=new Thread(  
            ()->{for(int i=0;i<100;i++) {  
                System.out.println("Girl");  
                try {  
                    Thread.sleep(10);  
                } catch (InterruptedException e) {  
                    // TODO Auto-generated catch block  
                    e.printStackTrace();  
                }  
            }  
        );  
        t1.start();  
        t2.start();  
    }  
}
```



Thread Management Methods

Method	Function
<code>start()</code>	Starts the thread and invokes its <code>run()</code> method.
<code>join()</code>	Waits for a thread to finish execution.
<code>sleep(long ms)</code>	Pauses the thread for a specified time.
<code>yield()</code>	Hints to the scheduler that the current thread is willing to yield CPU.
<code>interrupt()</code>	Interrupts a thread in a blocked or waiting state.
<code>setPriority(int priority)</code>	Sets the thread's priority (1 to 10, or <code>MIN_PRIORITY</code> , <code>NORM_PRIORITY</code> , <code>MAX_PRIORITY</code>).
<code>setDaemon(boolean on)</code>	Marks the thread as a daemon thread (background thread).



Atomic in Multithreading

- In Java, **atomic operations** are actions that cannot be interrupted or observed in an incomplete state by any thread. This is important when multiple threads are working on shared data, to prevent race conditions.
- A **race condition** in Java multithreading occurs when two or more threads access shared resources (such as variables, objects, or data structures) concurrently, and the final outcome depends on the unpredictable order in which the threads execute. This can lead to incorrect, inconsistent, or unexpected behavior in your program.



Example of a Non-Atomic Operation

Let's say you have a counter, and two threads are trying to increment its value.

```
public class Counter {  
    private int count = 0;  
  
    public void increment() {  
        count++; // Not atomic!  
    }  
  
    public int getCount() {  
        return count;  
    }  
}
```

The `count++` operation looks simple, but it is actually made up of three steps:

1. Read the current value of count.
2. Add 1 to the value.
3. Write the new value back to count.



Example of a Non-Atomic Operation

What Makes an Operation Atomic?

If two threads execute this at the same time:

- Thread 1 reads count as 0.
- Thread 2 also reads count as 0.
- Both threads increment it to 1 and write 1 back to count.

The result? You lose one increment because the operations overlapped.

An atomic operation ensures that no thread can interfere with it once it has started. This guarantees:

- The operation is completed fully or not at all.
- The operation is invisible to other threads until it's finished.



How to achieve the atomicity in java

- Use the **synchronized** keyword: By synchronizing the method or block, only one thread can execute it at a time.

```
public synchronized void increment() {  
    count++;  
}
```

- Use **java.util.concurrent.atomic** package: Java provides atomic classes like **AtomicInteger** to handle such operations easily.

```
import java.util.concurrent.atomic.AtomicInteger;  
  
public class Counter {  
    private AtomicInteger count = new AtomicInteger(0);  
  
    public void increment() {  
        count.incrementAndGet();  
    }  
  
    public int getCount() {  
        return count.get();  
    }  
}
```



What does join() do?

- The join() method in Java is used to pause the execution of the current thread until the thread on which join() was called has finished

```
public class AtomicMain {  
    public static void main(String[] args) {  
        // TODO Auto-generated method stub  
        Counter c=new Counter();  
        Thread t1=new Thread(  
            ()->{  
                for(int i=0;i<1000;i++) {  
                    c.increment();  
                }  
            }  
        );  
        Thread t2=new Thread(  
            ()->{  
                for(int i=0;i<1000;i++) {  
                    c.increment();  
                }  
            }  
        );  
        t1.start();  
        t2.start();  
        System.out.println("The value of count is "+c.getCount());  
    }  
}
```

In this example,
the value of
count is zero,
because Main is
not waiting t1
and t2 to finish



Example of using join()

```
public class AtomicMain {  
    public static void main(String[] args) {  
        // TODO Auto-generated method stub  
        Counter c=new Counter();  
        Thread t1=new Thread(  
            ()->{  
                for(int i=0;i<1000;i++) {  
                    c.increment();  
                }  
            });  
        Thread t2=new Thread(  
            ()->{  
                for(int i=0;i<1000;i++) {  
                    c.increment();  
                }  
            });  
        t1.start();  
        t2.start();  
        try {  
            t1.join();//waits for t1 thread to finish  
            t2.join();//waits for t1 thread to finish  
        } catch (InterruptedException e) {  
            // TODO Auto-generated catch block  
            e.printStackTrace();  
        }  
  
        System.out.println("The value of count is "+c.getCount());  
    }  
}
```

The value of
count is 2000



Threads communication

- In Java, **thread communication** refers to the mechanism by which threads can coordinate their actions by sharing data or signaling each other.
- The key methods for thread communication in Java are:
- **wait()**: Makes the current thread wait until another thread notifies it.
- **notify()**: Wakes up a single thread that is waiting on the object's monitor.
- **notifyAll()**: Wakes up all threads waiting on the object's monitor.

These methods must be called inside a synchronized block or method



Producer and consumer example

```
class Counter{
    int count;
    boolean valueSet=false;
    public synchronized void put(int count) {
        while(valueSet) {
            try {
                wait();
            } catch (InterruptedException e) {
                // TODO Auto-generated catch block
                e.printStackTrace();
            }
        }
        System.out.println("Put :"+count);
        this.count=count;
        valueSet=true;
        notify();
    }
    public synchronized void get() {
        while (!valueSet) {
            try {
                wait();
            } catch (InterruptedException e) {
                // TODO Auto-generated catch block
                e.printStackTrace();
            }
        }
        System.out.println("Get :"+count);
        valueSet=false;
        notify();
    }
}
```

```
class Producer implements Runnable{
    Counter count;

    public Producer(Counter count) {

        this.count = count;
        Thread t=new Thread(this, "Producer");
        t.start();
    }

    @Override
    public void run() {
        int i=0;
        while(true) {
            count.put(i++);
            try {
                Thread.sleep(1000);
            } catch (InterruptedException e) {
                // TODO Auto-generated catch block
                e.printStackTrace();
            }
        }
    }
}
```



Producer and consumer example

```
class Consumer implements Runnable{
    Counter count;

    public Consumer(Counter count) {

        this.count = count;
        Thread t=new Thread(this, "Consumer");
        t.start();
    }

    @Override
    public void run() {

        while(true) {
            count.get();
            try {
                Thread.sleep(1000);
            } catch (InterruptedException e) {
                // TODO Auto-generated catch block
                e.printStackTrace();
            }
        }
    }
}

public class ProducerConsumer {
    public static void main(String[]a ) {
        Counter c=new Counter();
        new Producer(c);
        new Consumer(c);
    }
}
```



Thread pool

- A thread pool is a collection of threads that are waiting to execute tasks. When a task is submitted, it is assigned to a thread from the pool. Once the task is done, the thread becomes available again to execute other tasks.
- Instead of creating a new thread every time you need to execute a task (which can be resource-intensive), a thread pool reuses a set of pre-created threads, making it more efficient.



Benefits of Thread Pool:

- **Reusability:** Threads in a pool are reused, reducing the overhead of creating and destroying threads.
- **Resource Management:** It manages a fixed number of threads, avoiding system overload caused by too many concurrent threads.
- **Better Performance:** Since threads are reused, the system doesn't need to repeatedly create and destroy threads, which improves performance.



Executor Framework

- The Executor Framework in Java is a part of the `java.util.concurrent` package, introduced in Java 5, to simplify multithreading and make it easier to manage threads. It provides tools to create, control, and manage threads in a more organized and flexible way.



Why Use the Executor Framework?

- Before the Executor Framework, managing threads in Java involved manually creating and managing Thread objects.
- This approach had drawbacks:
- Manually starting and stopping threads was error-prone.
- It was difficult to control the number of threads running at a time.
- Threads were not reusable, leading to performance issues.

The Executor Framework solves these problems by:

- Managing a pool of threads (called a thread pool).
- Reusing threads to avoid the overhead of creating new ones.
- Providing a clean interface for running tasks asynchronously.



Key Interfaces

- The Executor Framework has three main interfaces:

1. Executor : The base interface that provides a simple method to execute tasks

`void execute(Runnable command);`

It is implemented by higher-level classes like **ThreadPoolExecutor**.

2. ExecutorService :

- An advanced version of Executor that allows more control over thread execution.
- Provides methods to:
- Submit tasks and get results (submit).
- Shutdown the executor (shutdown or shutdownNow).
- Manage task completion (invokeAll or invokeAny).



Key Interfaces

3. **ScheduledExecutorService :**

- Extends ExecutorService to schedule tasks to run after a delay or repeatedly.



Examples of Executor framework

- https://drive.google.com/drive/folders/1ngBvMdFKZQVv2tIPpa-ZdAyOAfnliijV?usp=drive_link

